



# 10

## ANTI-DEBUGGING



*Anti-debugging* is a series of anti-reversing techniques used by malware (and even some legitimate programs) to hamper or prevent debugging. For example, the malware might try to interfere with the debugger process if it detects that it's attached to a debugger, or it might try to prevent debugging altogether by using so-called anti-attach mechanisms or crashing the debugger program. In this chapter, we'll explore some of these techniques in detail.

### Using Windows API Functions to Access the PEB

As you learned in [Chapter 1](#), the Process Environment Block (PEB) is a structure that contains pointers to information in memory about the currently running process. The PEB includes several pointers that are relevant to anti-debugging, as listed in [Table 10-1](#).

**Table 10-1:** PEB Members Relating to Anti-debugging

| PEB offset | PEB member    | Description                                                                            |
|------------|---------------|----------------------------------------------------------------------------------------|
| 0x002      | BeingDebugged | Indicates whether the program is currently being debugged                              |
| 0x018      | ProcessHeap   | Contains pointers to the heap's <code>Flags</code> and <code>ForceFlags</code> members |
| 0x068      | NtGlobalFlag  | Contains information related to the creation of memory heaps                           |

Windows exposes a wealth of data about its internal workings to programs via its APIs. Some Windows API functions have the sole purpose of telling the calling program whether a debugger is attached, and malware can easily exploit this fact by having them query the PEB to determine whether it's being debugged. Not only that, but malware can also abuse some API functions to trick Windows into exposing an attached debugger. In this section, we'll look at some common ways in which malware can leverage the Windows API and NT API to try to identify a malware analyst debugging its code. Later in the chapter, you'll see how malware can also query the PEB directly, which is why it's important to have at least a basic understanding of the PEB's relevant members.

### ***IsDebuggerPresent and CheckRemoteDebuggerPresent***

One of the best-known and simplest Windows functions for detecting debuggers is `IsDebuggerPresent`. This function returns a nonzero value if the current process is being debugged; otherwise, it returns 0. The `CheckRemoteDebuggerPresent` function returns the same information, but with a value of `True` or `False`. The following example shows how malware might use it:

```
--snip--  
push [ebp+hProcess]  
push [ebp+DebuggerPresent]  
call CheckRemoteDebuggerPresent  
--snip--
```

Two parameters are being pushed to the stack here: a handle to the target process (in this case, the malware's own process, `hProcess`) followed by a pointer to a variable that will receive the information returned (`DebuggerPresent`). Once the malware calls `CheckRemoteDebuggerPresent`, the return value (`True` or `False`) is stored in the `DebuggerPresent` variable. As you'll see later in the chapter, these are some of the easiest debugger detection techniques to circumvent.

### ***NtQueryInformationProcess***

To detect a debugger, `CheckRemoteDebuggerPresent` calls the system function `NtQueryInformationProcess`, a lower-level Windows NT function that can return a lot of different information about the system.

`NtQueryInformationProcess` can also be called directly. It takes several parameters, including `ProcessHandle` (a handle to the target process) and the `ProcessInformationClass` value (the type of information that should be returned). In the case of debugger detection, the `ProcessInformationClass` value would be 7, or `ProcessDebugPort`. A nonzero value indicates that the process is currently running under a debugger.

Alternatively, the malware author can specify `ProcessInformationClass` `ProcessDebugFlags` (1F). If the process is being debugged, the debug flags will be set, tipping off the malware. Finally, the `ProcessDebugObjectHandle` (1E) information class can also reveal a debugger.

### ***NtQuerySystemInformation***

`NtQuerySystemInformation` is a Windows NT function that can be used to, you guessed it, query system information. This function can return

many different types of information, and an especially crucial structure for our purposes is `SYSTEM_KERNEL_DEBUGGER_INFORMATION`. This structure contains two important values for debugger detection: `KdDebuggerEnabled` and `KdDebuggerNotPresent`, both of which can be used to detect a kernel debugger attached to the calling process. *Kernel debuggers* are specialized tools for debugging low-level software and sometimes for malware analysis. If `KdDebuggerEnabled` returns nonzero or `KdDebuggerNotPresent` returns 0, it might indicate to the malware that a kernel debugger is present.

### ***OutputDebugString***

The `OutputDebugString` function simply displays a string in the debugger. If a process is attached to a debugger, the string will be displayed; if not, `OutputDebugString` will return with an error. Malware can abuse this function by manually setting an error code using `SetLastError` (this error can be any arbitrary value), calling `OutputDebugString` with a random string value, and then calling `GetLastError` to check whether the error state has changed, as demonstrated in the following pseudocode:

```
SetLastError("5");
OutputDebugString("testing123");

error = GetLastError();

if (error == "5"):
    // Debugger detected.
    // Execute evasion code, such as TerminateProcess.
else:
    // Did not detect debugger; continue execution.
```

This malware sample sets an error code of 5 using `SetLastError`, then invokes `OutputDebugString` with the random string value `testing123`. Next, the malware calls `GetLastError` to get the last error code and compares that value to the error code from the previous `SetLastError` call. If the call to `OutputDebugString` succeeded with no error, the `GetLastError` code should

still be 5, meaning that a debugger is present. This is an old technique, but malware occasionally still attempts to use it.

### ***CloseHandle and NtClose***

If a malware sample is being debugged and the malware attempts to call the `CloseHandle` OR `NtClose` function with an invalid handle, it will raise an `EXCEPTION_INVALID_HANDLE` exception. Once this exception is triggered, it will be passed to the debugger's exception handler, tipping off the malware that it is being debugged. The following simple code demonstrates this:

```
mov ebx, [invalid_handle]
call NtClose
```

The preceding code calls `NtClose` with an invalid handle as a parameter. Once executed, this will raise an exception and tip off the malware that it is being debugged.

### ***NtQueryObject***

For a debugger to work properly, it must create a special kernel object called a *debug object*. Malware can call `NtQueryObject` to get a list of all debug objects, whose presence would indicate either that the malware is currently being debugged or that the host has used a debugger in the past.

The first parameter for this function, `Handle`, is the handle to the object being queried for information. The second parameter, `NtQueryObject`, accepts an `ObjectInformationClass` value that will tell the function what type of data needs to be returned. The third parameter, `ObjectInformation`, accepts a pointer to where the returned data will be stored.

If you spot malware calling `NtQueryObject`, providing an `ObjectInformationClass` value of 3 (`ObjectAllTypesInformation`), and then in-

specting the `ObjectInformation` buffer for strings such as `DebugObject`, you can be relatively certain the malware is attempting to identify a debugger.

## Heap Flags

The PEB contains pointers to the process's memory heap structure, known as the *process heap*, at location `0x18` (`0x30` for 64-bit processes). The process heap contains various members that are pointers to additional data; two of these members are `Flags` and `ForceFlags`, which point to a block of data that provides information to the Windows kernel about the process heap memory. In Windows 7 and above, if a process is being debugged, the value of the `Flags` member will be `0x40000062`, and the value of the `ForceFlags` member will be `0x40000060`. To detect a debugger, malware may attempt to read these values in its heap structure by calling the function `RtlQueryProcessHeapInformation` OR `RtlQueryProcessDebugInformation`, or it may manually read the PEB as we'll discuss in the next section.

This isn't a common anti-debugging technique, so I won't go into further detail here, but it's important to be aware that it exists. If you spot malware calling one or both of these functions, inspect the code that follows. If the malware is querying the `Flags` OR `ForceFlags` member, there's a good chance it's trying to detect your debugger.

## Directly Accessing the PEB

Rather than relying on the Windows functions described in the previous section, malware might directly access and read the PEB, as shown in the following assembly code:

```
--snip--  
mov eax, [fs:0x30]  
cmp [eax+0x2], 1
```

```
jnz DebuggerDetected
--snip--
```

Here, in order to obtain the address of the PEB to read it, the malware moves the address at `fs:0x30` into `eax`. (As you might recall from [Chapter 1](#), `fs:0x30` is the address for the beginning of the PEB.) Next, the malware compares the value of `1` to the value of `eax+0x2`, which is the `BeingDebugged` field in the PEB structure. If this value is `1`, it's likely that a debugger is attached to the process, and the malware might respond accordingly.

Another example of manual PEB reading involves the `NtGlobalFlag`, which is located at byte `0x68` in the PEB and contains information related to the creation of memory heaps. When a process is started under a debugger, the value of `NtGlobalFlag` will be `0x70`, so malware can read that value in its PEB for a quick debugger check.

Keep in mind that malware can query any member of the PEB structure in this manner. Accessing the PEB directly, rather than invoking common Windows functions in order to do so, can be an effective way for malware to detect a debugger without raising alarms.

## Timing Checks

[Chapter 7](#) described timing checks specifically in the context of detecting sandboxes and VM environments, but these checks can also be used to detect debuggers. Three of the common timing-based methods for debugger detection are `GetTickCount`, the `rdtsc` instruction, and system time checks.

As you might remember from [Chapter 5](#), `GetTickCount` returns the number of milliseconds that have elapsed since system boot-up. Malware can call `GetTickCount` at various points in its code to see how much time has elapsed since its last call to this function. When malware is being debugged, the program will naturally execute more slowly, especially if the malware analyst is setting breakpoints throughout the debugging process. If the analyst happens to place a breakpoint on a function or is

single-stepping through code, the difference between the first call of `GetTickCount` and the last will be much greater than it would be otherwise. Malware can exploit this fact to detect a debugger.

Malware can use the `rdtsc` instruction in a similar manner. By placing `rdtsc` instructions throughout its code, malware can determine whether it's being debugged based on CPU timing (see the x64dbg screenshot in [Figure 10-1](#)).

|          |               |                                       |
|----------|---------------|---------------------------------------|
| 00402705 | 57            | push edi                              |
| 00402706 | 33DB          | xor ebx,ebx                           |
| 00402708 | 0F31          | <b>rdtsc</b>                          |
| 0040270A | 8BF8          | mov edi,eax                           |
| 0040270C | 8955 FC       | mov dword ptr ss:[ebp-4],edx          |
| 0040270F | FF15 6C704000 | call dword ptr ds:[<&GetProcessHeap>] |
| 00402715 | 0F31          | <b>rdtsc</b>                          |
| 00402717 | 6A 00         | push 0                                |
| 00402719 | 8BF0          | mov esi,eax                           |
| 0040271B | 8955 FC       | mov dword ptr ss:[ebp-4],edx          |
| 0040271E | FF15 98704000 | call dword ptr ds:[<&CloseHandle>]    |
| 00402724 | 0F31          | <b>rdtsc</b>                          |
| 00402726 | 8955 FC       | mov dword ptr ss:[ebp-4],edx          |

Figure 10-1: Malware using `rdtsc` as an anti-debugging technique

Finally, malware can simply query the system time by calling functions such as `GetLocalTime`, `GetSystemTime`, and `NtQuerySystemTime` at various points in its code to detect a debugger in use.

There are many other methods of using timing checks to catch a debugger in its tracks; these are only some of the most common. The takeaway here is that if you spot malware that is occasionally calling timing functions or using instructions such as `rdtsc` throughout its code, it could be using anti-debugging techniques. The best way to address this is to simply avoid using a debugger. However, this isn't always practical. You may have to identify where these functions and instructions are being executed in the malware and patch them out, or you may modify their return values to trick the malware into believing it's not being debugged.

## System Artifacts

Malware may be able to detect a debugger via system artifacts such as desktop windows, registry keys, and loaded modules. We'll cover each



briefly in this section, but you might also find it helpful to refer back to [Part II](#), which covered how malware might search for these types of artifacts.

### ***Hunting for Debugger Windows***

[Chapter 5](#) discussed how malware can detect analyst tools using functions such as `FindWindow`, which locates a desktop window, and `EnumWindows`, which enumerates open desktop windows. Malware can also use these functions to look specifically for debugger windows. For example, it might call `FindWindow` to enumerate open windows with debugger product names such as `x64dbg`, `OllyDbg`, or Immunity debugger. [Figure 10-2](#) shows the ransomware variant Satan hunting for any open `OllyDbg` windows.

|   |          |                 |                 |                                                      |
|---|----------|-----------------|-----------------|------------------------------------------------------|
| • | 00402DA2 | 56              | push esi        | eax:L"OLLYDBG"<br>call FindWindowW<br>eax:L"OLLYDBG" |
| • | 00402DA3 | 50              | push eax        |                                                      |
| • | 00402DA4 | FFD7            | call edi        |                                                      |
| • | 00402DA6 | 85C0            | test eax,eax    |                                                      |
| • | 00402DA8 | v 0F85 83020000 | jne evil.403061 |                                                      |

Figure 10-2: The malware Satan looking for OllyDbg-related windows

### ***Enumerating Loaded Modules***

When you attach malware to a debugger, the debugger may load modules into the malware's address space; for example, the WinDbg debugger may load the `dbghelp.dll` library. Malware can locate a suspect module by calling the `GetModuleHandle` function and passing the module's name (such as `dbghelp.dll`) as a parameter. Alternatively, it can enumerate all loaded modules by using `Module32First` and `Module32Next` and then search for a specific module name.

### ***Searching for Debugger Processes***

Malware can also enumerate the running processes on the host to search for debugger processes. To do so, it might call `CreateToolhelp32Snapshot`, `Process32First`, and `Process32Next`, then look specifically for common debugger process names, such as `ollydbg.exe`, `x64dbg.exe`, or `ida64.exe`.

## Checking Parent Processes

To detect a debugger, malware can check to see what its parent process is. If the malware is running inside a debugger, its parent process will be the debugger process. If the malware detects it is running as a child process of a debugger process (such as *x64dbg.exe*), the malware will identify that it is being debugged.

Malware can detect its parent process in a few ways. One approach is to obtain its own process ID (`GetCurrentProcessId`), take a snapshot of all running processes (`CreateToolhelp32Snapshot`), and use `Process32First` and `Process32Next` to search for its own process name. Once it finds its process, the process snapshot structure contains an entry (`th32ParentProcessID`) representing its parent process ID.

Similarly, malware could spawn a child process using `CreateProcess`, making the malware's original process the parent process of this new child. The child process could then invoke `DebugActiveProcess` with its parent process as a parameter. If the parent process (the original malware process) is already being debugged, this function will throw an exception such as `STATUS_PORT_ALREADY_SET`, cluing in the malware that there's a debugger attached to it.

## Breakpoint Detection and Traps

When investigating malware in a debugger, a malware analyst commonly creates breakpoints on specific instructions, function calls, or specific memory segments. As you might remember from [Chapter 3](#), the act of creating software breakpoints in the malware's code modifies the running malware sample. This means that the malware can detect these breakpoints in some interesting ways. This section will discuss some of the breakpoint detection, circumvention, and exploitation methods malware might employ.

## Detecting Debuggers with Breakpoints

When a debugger hits a breakpoint instruction such as `int 3` (one of the most common) in a program, it breaks at that point in the program's code. This occurs because the breakpoint instruction causes an interrupt exception in the program, which ultimately transfers control to the debugger. When a program is not being debugged, however, breakpoint instructions cause an `EXCEPTION_BREAKPOINT` and the control flow is passed to the program's default exception handler.

This is a great way for malware to test whether it's being debugged. If malware executes the instruction `int 3` and it's not being debugged, an `EXCEPTION_BREAKPOINT` will be raised and the exception handler will be invoked. Of course, the opposite is also true. If this `EXCEPTION_BREAKPOINT` doesn't invoke the exception handler, the malware can infer that it's being debugged. Take a look at the following simplified pseudocode:

```
--snip--
IsBeingDebugged() {
    try {
        ❶ asm ("int 3");
        return true;
    }
    ❷ catch (EXCEPTION_EXECUTE_HANDLER) {
        return false;
    }
}
--snip--
```

This pseudocode features a simple `try... catch` statement. The malware attempts to execute the `int 3` instruction ❶, and if this statement successfully returns without invoking an exception handler, the malware assumes it's running in a debugger. If this instruction raises an exception ❷, however, the malware can safely assume it's not attached to a debugger.

## Detecting and Circumventing Software Breakpoints

Malware can also directly detect the use of breakpoints by implementing breakpoint-scanning techniques. In the following assembly code example, the malware scans its code for the breakpoint instruction `int 3` (`0xCC` in hex):

```
--snip--  
mov ebx, <kernel32.WriteProcessMemory>  
cmp byte ptr ds:[ebx], 0xCC  
--snip--
```

This malware sample is trying to determine whether a malware analyst has set a software breakpoint on the `WriteProcessMemory` function. First, the malware moves the address of `WriteProcessMemory` into the `ebx` register. Then, it compares the value of `0xCC` (the `int 3` debugger breakpoint instruction) to the first byte in `ebx`, which is the beginning of the `WriteProcessMemory` function and where the breakpoint will reside. Once the malware identifies a software breakpoint, it may attempt to overwrite or clear the breakpoint.

You can overcome some breakpoint detection techniques by using uncommon breakpoint instructions. Many debuggers have this option. In `x64dbg`, simply navigate to **Options ▸ Settings ▸ Engine** and set your preferred breakpoint instruction under **Default Breakpoint Type**, as shown in [Figure 10-3](#).

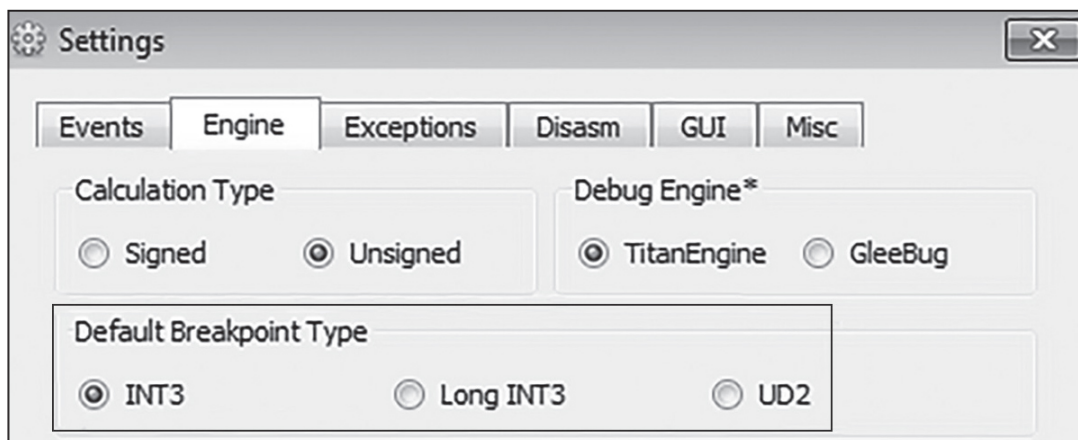


Figure 10-3: Setting the default breakpoint type in x64dbg

Some variations of malware look for these alternative breakpoint instructions as well, so a better option is to use hardware breakpoints to circumvent software breakpoint detection techniques. Hardware breakpoints can also be circumvented by malware, however, as the next section will discuss.

### ***Detecting and Circumventing Hardware and Memory Breakpoints***

Like software breakpoints, hardware breakpoints can be used by analysts to intercept function calls, break on interesting behavior, and generally control the malware's execution. Because hardware breakpoints are implemented in CPU registers (DR0–DR3) rather than as instructions, malware can scan these registers for them. If any of these registers contain data (specifically, a memory address), the malware might assume there's a hardware breakpoint in place and take evasive actions, such as clearing these registers, effectively removing the breakpoints. One way to find hardware breakpoints is by using the `GetThreadContext` function (or `Wow64GetThreadContext` for 64-bit programs) as follows:

```
CONTEXT context;  
context.ContextFlags = CONTEXT_DEBUG_REGISTERS;  
HANDLE hThread = GetCurrentThread();  
GetThreadContext(hThread, &context);
```

```
if ((context.Dr0) || (context.Dr1) || (context.Dr2) || (context.Dr3)) {  
    return true;  
}
```

In this sample pseudocode (adapted from <https://www.codeproject.com/Articles/30815/An-Anti-Reverse-Engineering-Guide>), the malware defines a new `CONTEXT` structure (`context`), which is a Windows structure used to store CPU state and related data for the process. This structure must be in place to store the debug register (DR0–DR3) data that the malware will check for shortly. Next, the malware specifies that the `CONTEXT_DEBUG_REGISTERS` should be the data that `GetThreadContext` returns. The malware then gets a handle to its current thread (`HANDLE`) and calls `GetThreadContext`, which will store the current thread's `CONTEXT_DEBUG_REGISTERS` in the `context` structure. Finally, the malware sample inspects the debug registers for data. If these registers are nonzero, the check returns true, informing the malware that a hardware breakpoint has been set by the debugger.

The malware can then completely remove any hardware breakpoints it detects by using `SetThreadContext` (OR `Wow64SetThreadContext` for 64-bit malware). Adding this line to the preceding code example will effectively clear the debug registers, subsequently removing the malware analyst's hardware breakpoints:

```
context.Dr0 = null;  
context.Dr1 = null;  
context.Dr2 = null;  
context.Dr3 = null;  
  
SetThreadContext(hThread, &context);
```

Finally, malware can detect memory breakpoints with the `ReadProcessMemory` function. Setting a memory breakpoint alters the memory page, so if the malware invokes `ReadProcessMemory` on a suspect memory page, and if it returns an unexpected value such as `PAGE_NOACCESS` or `PAGE_GUARD`, it might infer that a hardware breakpoint has been set on

this memory page. Another option to achieve the same effect is for the malware to execute `VirtualQuery`, `VirtualQueryEx`, or `NtQueryVirtualMemory`. We'll discuss memory breakpoints more in the next section.

If you suspect that a malware sample is using any of the techniques described in this section, it's helpful to hook (place a breakpoint on) these functions. Once a breakpoint is hit, you can simply modify the return value or `nop` out the call to the function altogether. The debugger plug-in `ScyllaHide`, described on [page 181](#), can also be useful here.

### ***Using Memory Page Guards for Breakpoint Detection***

*Memory page guards* are a special flag implemented in Windows for memory access. When allocating new memory pages, a program can add the `PAGE_GUARD` flag as a sort of alarm system for memory access. When this area of memory is accessed by the program (or by a different program), it raises a `STATUS_GUARD_PAGE_VIOLATION` exception. If the program is running under a debugger, `PAGE_GUARD` often behaves a bit differently (because the debugger is handling the exception) and won't raise the normal exception.

Malware can take advantage of this by implementing page guards on certain memory pages. If these pages are accessed from outside a debugger (meaning that the malware isn't being debugged), the malware will raise the exception as normal. If no exception is raised, depending on the debugger being used and its configuration, this could alert the malware that it's being debugged. To set page guards in memory, malware can call `VirtualProtect` with the `PAGE_GUARD` (`0x100`) value set. The following code shows a malware sample calling `VirtualAlloc` with this parameter set:

```
--snip--
push 104h ; 0x100 (PAGE_GUARD) + 0x4 (READ/WRITE)
push edi
push esi
```



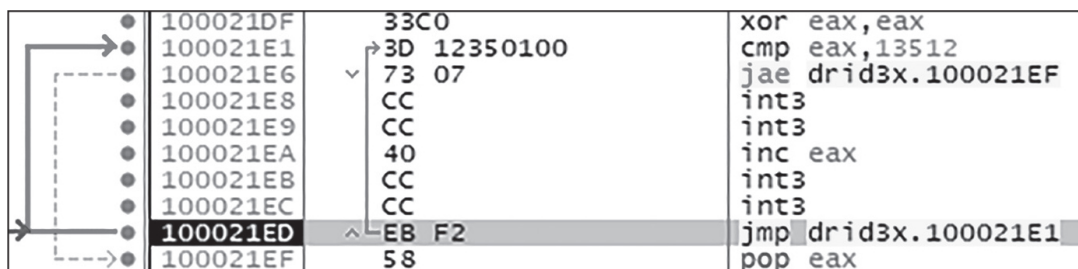
```
call VirtualAlloc
--snip--
```

This code calls `VirtualAlloc` with the `flNewProtect` value set to `0x104`, which is a combination of `0x100` (`PAGE_GUARD`) and `0x4` (`READ/WRITE`) protection. Memory allocation and `VirtualAlloc` are covered in more detail in [Part IV](#), particularly [Chapter 17](#).

### Using Breakpoint Traps

Malware can insert breakpoint instructions such as `int 3` and `int 2d` throughout its code in order to force the debugger to break often, causing a lot of trouble for the malware analysts debugging it. A constantly breaking debugger makes debugging a headache. Breakpoint instructions intentionally placed in malware are sometimes called *traps*.

[Figure 10-4](#) shows a sample of the Dridex malware in x64dbg using the `int 3` trap technique.



|          |             |                     |
|----------|-------------|---------------------|
| 100021DF | 33C0        | xor eax, eax        |
| 100021E1 | 3D 12350100 | cmp eax, 13512      |
| 100021E6 | 73 07       | jae drid3x.100021EF |
| 100021E8 | CC          | int3                |
| 100021E9 | CC          | int3                |
| 100021EA | 40          | inc eax             |
| 100021EB | CC          | int3                |
| 100021EC | CC          | int3                |
| 100021ED | EB F2       | jmp drid3x.100021E1 |
| 100021EF | 58          | pop eax             |

Figure 10-4: A Dridex malware sample using a breakpoint anti-debug trap

The Dridex code shown here executes two `int 3` instructions, increments `eax`, executes two more `int 3` instructions, and then continues to loop. This function repeats 13,512 times (`cmp eax, 13512`), which can cause major frustration for the malware analyst debugging the sample.

It's often difficult to circumvent this trap technique in malware. The best approach is to identify the problematic instructions and patch them out. As in the case of this Dridex sample, the malware may create a loop (using `jmp`, `jnz`, `jz`, and so on) of these breakpoint instructions, in



which case you'd need to patch out or modify the loop instruction to bypass this behavior.

## Unhandled Exceptions

Malware can set a top-level exception handler (called an *unhandled exception filter*) to test for a debugger. This supersedes any other handlers, such as the default SEH, by first invoking `SetUnhandledExceptionFilter`, which allows the calling program to become the top-level exception handler, then invoking `UnhandledExceptionFilter`, which forces the exception handler to execute. When a malware program isn't being debugged, the exception will be passed to the new handler. If it is being debugged, then the new handler won't be called and the exception will be passed to the debugger, indicating to the malware that it's running inside a debugger.

To deal with this technique, you can intercept calls to both `SetUnhandledExceptionFilter` and `UnhandledExceptionFilter` (via breakpoints or hooks) and modify the function calls or patch them out of the code completely. [Chapter 11](#) discusses exception handling in more detail.

## Checksums, Section Hashing, and Self-Healing

Malware can take a checksum (or hash) of its code to determine whether a breakpoint has been placed or an analyst has otherwise modified the code in the debugger. Using any hashing algorithm (MD5, for example), the malware author can create a hash of its code line by line or hash an entire section of code (typically called *section hashing*), then compare this with a baseline sum. If there's a discrepancy, the malware may assume that a breakpoint has been set or its code has been otherwise modified.

Malware can also implement so-called *self-healing* techniques, in which it stores clean copies of its code and, if it detects code tampering, re-

stores the original version. The following pseudocode demonstrates how this might look in practice:

```
// Calculate the checksum of the clean code.
clean_code_checksum = calculate_checksum(clean_code)

// At runtime, recalculate the checksum and compare it to the stored value.
malware_code = read_malware_code()
malware_code_checksum = calculate_checksum(malware_code)

// If checksums do not match, terminate the malware.
if malware_code_checksum != clean_code_checksum:
    terminate_malware()
else:
    // The code has not been tampered with! Continue running.
```

There's not much an analyst can do to circumvent this technique. The most effective method is to identify where the malware is obtaining a checksum of a certain segment of code and then patch this functionality out of its code. Hardware breakpoints can also be used in the event that the malware is using this approach to look for software breakpoints.

## Exploiting, Crashing, and Interfering with the Debugger

Sometimes the most effective anti-debugging method is to directly interfere with the debugger by causing it to crash or behave unpredictably. Just like any software, debuggers can have bugs in their code that allow the malware to interfere with analysis, crash the debugger, or possibly even crash the operating system itself. A popular example is a bug in version 1.1 of OllyDbg whereby malware could call the `OutputDebugString` function and pass `%s` as a parameter; OllyDbg would be unable to handle this value, causing the running malware sample to crash and preventing further debugging.

Perhaps slightly less offensive is the `BlockInput` function, which malware can abuse to interfere with analysis tools. `BlockInput` takes only one pa-

parameter: `fBlockIt`. If this parameter is set to 1, mouse and keyboard events to all applications will be blocked, which can seriously interfere with the debugging process. If malware detects that it is being manually inspected or running under a debugger, it may simply call `BlockInput` as a self-defense mechanism. Luckily, you can easily get around this technique by using CTRL-ALT-DELETE, which will escape the `BlockInput` routine. You can also modify the function call in a debugger, setting the `fBlockIt` parameter to 0 instead of 1.

Finally, malware can use `NtSetInformationThread` to hide code execution from the debugger or to crash the debugger altogether. By passing the `ThreadInformationClass` value `ThreadHideFromDebugger` (in hex, 0x11) to the `NtSetInformationThread` function, the malware can covertly execute code, causing the malware analyst to lose control of the running sample. Additionally, if the malware executes a breakpoint instruction (such as `int 3`) in the covertly executed code, the malware process will hang indefinitely in the debugger. I'll discuss this technique in the context of covert code execution in [Chapter 11](#).

These are only a few examples of known debugger exploitation techniques, and there are certainly more that are unknown. Malware analysts should be wary of these techniques and expect new ones to pop up when analyzing advanced evasive malware. Remember to always conduct analysis and debugging in a safe testing environment.

## Debug Blocking and Anti-attach Techniques

Instead of detecting a debugger or actively interfering with it, malware may simply try to prevent it from functioning altogether. This series of techniques is often called *debug blocking* or *anti-attach*.

To attach to a running malware sample, normally the debugger calls the Windows function `DebugActiveProcess`. In the event that the process is already being debugged, this function will fail. Malware can take advantage of this by simply acting as its own debugger. It can spawn a child process and set the parent process as the debugger. When attempting to

attach a debugger to this child process, a malware analyst will be met with a frustrating `STATUS_PORT_ALREADY_SET` exception, meaning that the process is already being debugged.

To overcome this technique, you can attach a debugger to the parent process and set a breakpoint on `WaitForDebugEvent`. When the breakpoint is hit, you can force the process (which is acting as the debugger) to unattach from its child process by invoking the `DebugActiveProcessStop` function.

#### NOTE

*For more information on this technique, check out the research paper “The Art of Unpacking” by Mark Vincent Yason. The paper is old but still very relevant. Read it at <https://www.blackhat.com/presentations/bh-usa-07/Yason/Whitepaper/bh-usa-07-yason-WP.pdf>.*

Malware can also modify common debugger functions to prevent debugging; the malware family GuLoader does exactly this. When attaching to a process, the debugger will invoke the functions `DbgBreakPoint` and `DbgUiRemoteBreakin`, which essentially allow the debugger to use breakpoints within the program being debugged. GuLoader modifies `DbgBreakPoint` by removing the `int 3` opcode and replacing it with `nop` instructions, effectively impairing the function. Likewise, GuLoader modifies the `DbgUiRemoteBreakin` function with an invalid call instruction, which will ultimately cause an exception, possibly resulting in a crash of the malware being debugged. *CrowdStrike* wrote an exceptional article on GuLoader; for more information on these techniques, read the paper at <https://www.crowdstrike.com/blog/guloder-malware-analysis/>.

## Other Anti-debugging Techniques

There are many more methods malware can use to discover or interfere with a debugger in use. And, of course, researchers and malware authors constantly identify new, creative ways to detect and circumvent

debuggers. I've tried to cover as many commonly used or particularly interesting techniques as possible in this chapter.

Some methods I chose not to describe here because they're uncommon or difficult to implement. For example, some malware families (rootkits, for instance) can directly inspect their `EPROCESS` blocks for signs of an attached debugger, but this is quite rare. Other techniques are subject to error. For example, malware can check its debug privileges by attempting to call `OpenProcess` with the target of a system process, such as `csrss.exe`. If the malware can get a handle to this process, it may infer that it's being debugged, but it's not always accurate. Another example is the use of the *trap flag*, a special flag in the `EFLAGS` and `RFLAGS` registers that can give away the presence of a debugger. Both methods can produce false positives, are more difficult to implement, and thus are not as widely used.

To end this chapter, we'll take a look at how to counter anti-debugging techniques that you encounter when reverse engineering.

## Countering Anti-debugging Techniques

In order to effectively bypass anti-debugging techniques, first you need to have an idea of what exactly you're up against. You could identify the techniques a malware sample is employing simply by single-stepping through its code and manually searching for them, but of course, this isn't very efficient. Typically, you'll want to use a disassembler in combination with a debugger to better understand any anti-debugging methods you encounter.

Before digging into any malware, I always inspect the sample in a PE static analysis and triage tool such as PEStudio or PE-bear. These types of tools allow me to inspect imports and strings in the file that may help identify possible anti-debugging-related libraries and function calls. I also use tools like CAPA, which I described in [Chapter 3](#).

Once I've identified anti-debugging techniques in the executable, I inspect the surrounding code with the help of a disassembler and decide how I'll counter and bypass these techniques. This often involves setting breakpoints on suspect code and function calls in the debugger and modifying the code dynamically.

There are some great tools that can help automate the process of bypassing anti-debugging techniques. *ScyllaHide* (<https://github.com/x64dbg/ScyllaHide>), a plug-in for x64dbg and other debuggers, is perhaps the most popular. It can hide a debugger from malware by dynamically modifying the malware's code and hooking suspect functions to bypass many debugger detection and anti-debugger techniques (as shown in [Figure 10-5](#)).

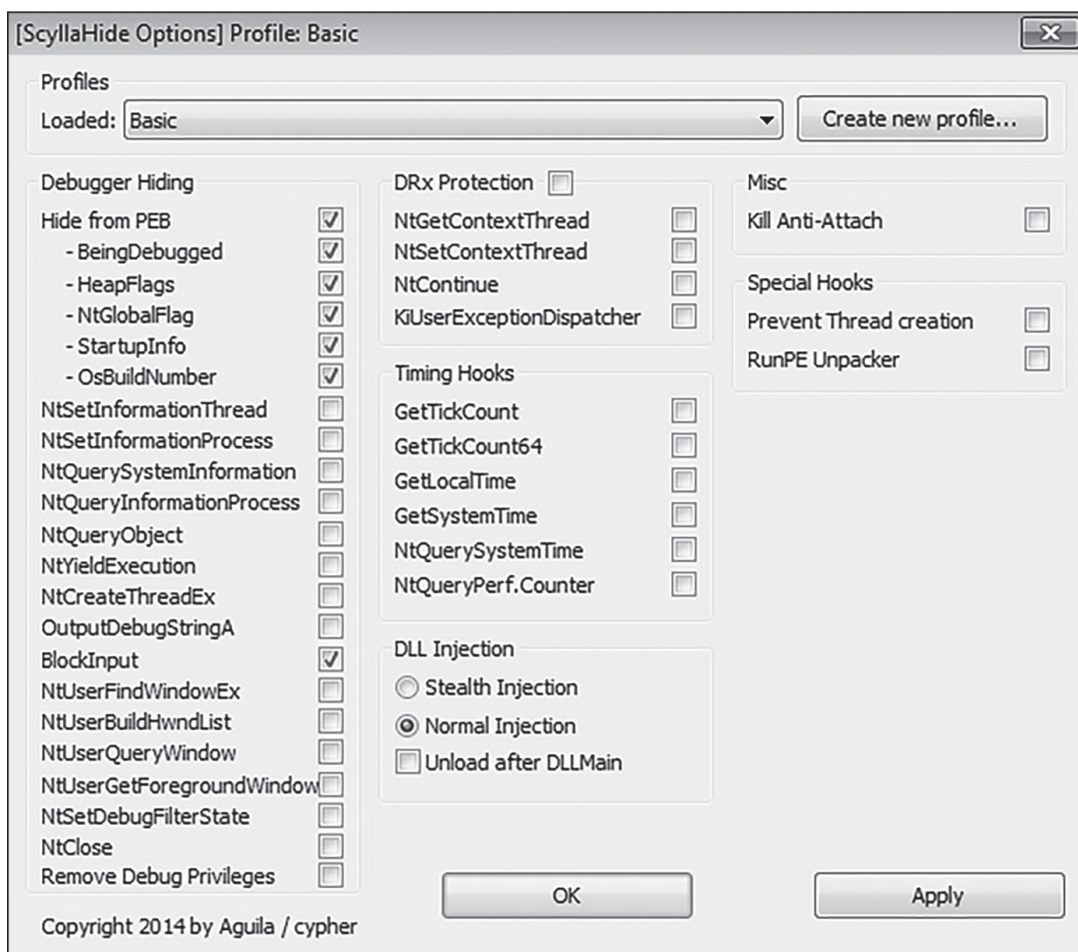


Figure 10-5: The ScyllaHide menu in x64dbg

To enable an *anti-anti-debugging* feature in ScyllaHide, simply check the box next to the feature, then select **Apply** and **OK**. Mousing over each

option will pop up more information about it. I often enable the entire left column of options, and this rarely causes problems. That said, while most of these options can be enabled safely, some of them may break the malware sample, possibly causing it to crash or behave in unexpected ways. So, use this with care.

*HyperHide* (<https://github.com/Air14/HyperHide>) provides many of the same features as *ScyllaHide*, but it's always a good idea to have multiple tools in your toolbox. **Figure 10-6** shows *HyperHide*'s anti-anti-debugging capabilities.



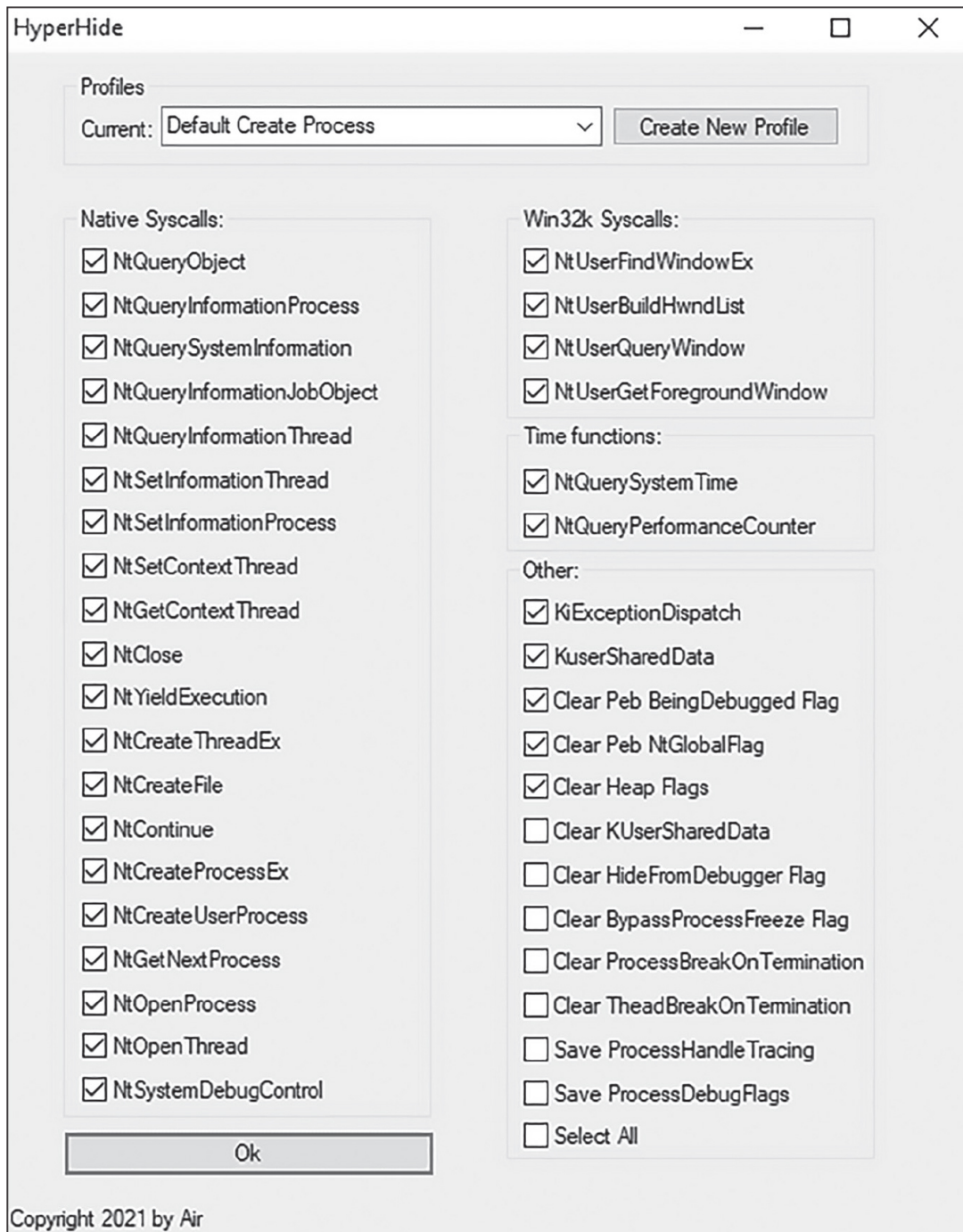


Figure 10-6: The HyperHide menu in x64dbg

Both tools feature very similar anti-anti-debugging options, but one may work better than the other in certain circumstances. Try them both out and see which you prefer.

## Summary

This chapter discussed many common anti-debugging methods that malware might use to detect and circumvent debugging tools. Many of

these techniques are widely used in all sorts of malware, from commodity infostealers to advanced bespoke and targeted threats, so it's important to understand the concepts described here. In the next chapter, you'll learn how malware can covertly execute code to evade dynamic analysis tools like debuggers and use misdirection techniques to disrupt the analysis process.